Unidad 1. Clases, objetos y encapsulamiento

Programación Orientada a Objetos · Facultad de Ingeniería · Notas de clase para uso académico

1. Del problema al modelo

La programación orientada a objetos organiza un programa alrededor de las cosas del dominio y no alrededor de los pasos. Si el problema es una biblioteca, aparecen libros, socios y préstamos. Cada una de esas ideas se convierte en una clase: una plantilla que describe qué datos guarda (atributos) y qué sabe hacer (métodos). Un objeto es un ejemplar concreto creado a partir de esa plantilla: el libro "Cien años de soledad" con código B-0412 es un objeto de la clase Libro.

Dos objetos de la misma clase comparten el comportamiento, pero cada uno tiene su propio estado. Cambiar la disponibilidad de un libro no altera la de otro. Esa separación es la que permite razonar sobre programas grandes por partes.

2. Atributos, métodos y constructores

El constructor deja al objeto en un estado válido desde el primer momento. Es el lugar para exigir los datos obligatorios y rechazar valores imposibles, en lugar de confiar en que quien use la clase recuerde llamar a varios métodos en orden.

```
public class CuentaAhorros {
    private final String numero;
    private double saldo;

    public CuentaAhorros(String numero, double saldoInicial) {
        if (saldoInicial < 0) throw new IllegalArgumentException("Saldo negativo");
        this.numero = numero;
        this.saldo = saldoInicial;
    }

    public void retirar(double valor) {
        if (valor <= 0 || valor > saldo) throw new IllegalArgumentException("Retiro inválido");
        saldo -= valor;
    }

    public double getSaldo() { return saldo; }
}
```

3. Encapsulamiento

Encapsular es ocultar el estado interno y exponer solo operaciones con sentido para el dominio. En el ejemplo, saldo es privado: nadie puede escribir cuenta.saldo = -500. La única forma de disminuirlo es retirar(), que valida la regla de negocio. Así la regla vive en un solo lugar.

- private: solo la propia clase accede al miembro.
- protected: la clase, sus subclases y el mismo paquete (en Java).
- public: cualquier clase puede usarlo; es el contrato que se promete mantener.

- Regla práctica: atributos privados, métodos públicos solo si otro objeto los necesita.

Un error frecuente es generar getters y setters para todos los atributos. Un setter público de saldo deshace el encapsulamiento. Conviene preguntarse qué operación real necesita el dominio: consignar, retirar o transferir, no "cambiar el saldo".

4. Objetos que colaboran

Un programa orientado a objetos funciona como un equipo: cada objeto hace una parte y delega el resto. Un Prestamo conoce el Libro y el Socio involucrados y le pide al libro que se marque como prestado; no modifica sus atributos directamente. A esto se le llama enviar mensajes.

5. Ejercicios de práctica

1. Modela la clase Estudiante con código, nombre y una lista de notas. ¿Qué validaciones pondrías en el constructor?
2. Agrega a CuentaAhorros un método transferir(destino, valor). ¿Por qué no debería modificar directamente el saldo del destino?
3. Explica con un ejemplo propio la diferencia entre una clase y un objeto.
4. Identifica en el código anterior qué pasaría si saldo fuera público.

Fuentes consultadas

• Oracle. The Java Tutorials: Object-Oriented Programming Concepts. docs.oracle.com/javase/tutorial/java/concepts
• Wikipedia. Programación orientada a objetos (CC BY-SA 4.0). es.wikipedia.org
Texto original redactado para el curso a partir de las fuentes citadas.